

Session 1

Session 2

Session 3

[🏠 Workshop Home](#) > [Day 1](#) > [Day 2](#) > [Day 3](#)

Day 3 — Multi-Agent AI & Portfolio

Build collaborative AI teams with CrewAI, develop your capstone portfolio project, present it live, and get career guidance to land AI roles.

 9:00 AM – 5:00 PM

3 Sessions

Capstone + GitHub Portfolio

CrewAI · Streamlit · Career Guidance

SESSION 1 OF 3

 9:00 – 11:00 AM

CrewAI — Multi-Agent Systems

📖 CORE CONCEPTS



Intuitive Analogy — Multi-Agent Engineering Teams

CrewAI structures multi-agent collaboration like a **specialized software engineering team**

👥. Specialized agents take on roles (e.g., Project Manager, Researcher, Developer, QA) with distinct goals and tool sets to execute complex workflows.



Why Multi-Agent Systems?

One AI agent trying to do everything gets confused and makes mistakes — just like one person trying to be a developer, designer, and manager simultaneously. Multi-agent systems divide work among specialized agents that collaborate, similar to a real software company.



Agent = Role + Goal + Backstory

Each CrewAI agent has three defining attributes:

- › **Role** — what the agent IS (e.g., "Senior Python Developer")
- › **Goal** — what it's trying to achieve
- › **Backstory** — personality and expertise context
- › **Tools** — optional capabilities (search, code, etc.)



Task = Work + Expected Output

- › **Description** — detailed instructions for the agent
- › **Expected Output** — what should be produced
- › **Agent** — which agent executes this task
- › **Context** — can use output of previous tasks
- › **Output File** — save result to file automatically



Crew = Team Orchestrator

- › Brings agents and tasks together
- › Manages execution order
- › `Process.sequential` — one after another
- › `Process.hierarchical` — manager delegates to agents
- › `verbose=True` — see agents think and collaborate



Sequential vs Hierarchical

FEATURE	SEQUENTIAL	HIERARCHICAL
Order	Fixed 1→2→3	Manager decides
Delegation	No	Yes
Flexibility	Low	High
Best for	Pipelines	Complex projects



Real-World Use Cases

- > 🧪 Research → Write → Review workflow
- > 💻 Plan → Code → Test → Document cycle
- > 📊 Collect data → Analyze → Report
- > ✉️ Draft email → Review → Send
- > 🛍️ Product research → Compare → Recommend



CrewAI Architecture

Define Agents
(Role+Goal+Backstory)



Define Tasks (What to do)



Create Crew (Team + Process)



kickoff(inputs) → Result

⚡ PROJECT: AI SOFTWARE COMPANY WITH CREWAI



Build an AI Software Company

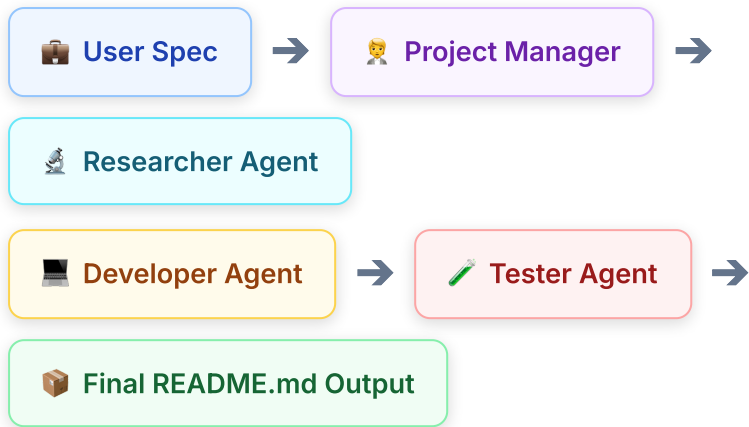
5 specialized agents collaborating to build software: Researcher → Developer → Tester → Doc Writer



What you'll build

A multi-agent team where a Project Manager coordinates a Researcher, Developer, Tester, and Documentation Writer to analyze a requirement, write code, test it, and produce documentation — all automatically.

PROGRAM ARCHITECTURE — CREWAI MULTI-AGENT TEAM



1

Install CrewAI

BASH

```
pip install crewai crewai-tools streamlit
```

PYTHON

ai_software_company.py

```
# Day 3 - Session 1: AI Software Company with CrewAI
import os
from crewai import Agent, Task, Crew, Process, LLM

# Configure Ollama LLM backend for CrewAI
local_llm = LLM(
    model="ollama/llama3.2",
    base_url="http://localhost:11434"
)

# — 1. Define the Agents —

project_manager = Agent(
    role="Project Manager",
    goal="Break down software requirements into clear tasks and",
    backstory="""You are an experienced software project manager with years
of experience. You excel at understanding requirements and creating
structured development workflows. You're organized, clear, and detail-oriented."
    llm=local_llm,
    verbose=True,
    allow_delegation=False
)

researcher = Agent(
    role="Senior Technical Researcher",
    goal="Research best approaches, algorithms, and design patterns for",
    backstory="""You are a senior software architect who loves exploring new
approaches to solve problems. You know multiple programming languages,
design patterns, and best practices. You provide thorough analysis and recommendations."
    llm=local_llm,
    verbose=True,
    allow_delegation=False
)

developer = Agent(
    role="Senior Python Developer",
    goal="Write clean, efficient, well-commented Python code based on requirements",
    backstory="""You are a senior Python developer with extensive experience in
production-quality code. You follow PEP 8 standards, add proper documentation,
handle edge cases, and write readable, maintainable code."
    llm=local_llm,
    verbose=True,
    allow_delegation=False
)
```

```

tester = Agent(
    role="QA Engineer",
    goal="Review the code, identify bugs, and write comprehensive test cases.",
    backstory="""You are a meticulous QA engineer who thinks about edge cases,
    input validation, and failure scenarios. You write pytest tests and
    provide specific, actionable feedback on code quality."""
    llm=local_llm,
    verbose=True,
    allow_delegation=False
)

```

```

doc_writer = Agent(
    role="Technical Documentation Writer",
    goal="Create clear, comprehensive documentation including READMEs and API docs.",
    backstory="""You write excellent technical documentation that is easy to follow.
    You include installation instructions, usage examples, API references,
    and troubleshooting tips. Your READMEs are always GitHub-ready."""
    llm=local_llm,
    verbose=True,
    allow_delegation=False
)

```

— 2. Define the Tasks —

```

def create_tasks(project_requirement: str):

    plan_task = Task(
        description=f"""Analyze this software requirement and create a structured project plan.

        Requirement: {project_requirement}

        Create:
        1. Clear understanding of what needs to be built
        2. List of key features to implement
        3. Suggested data structures or classes needed
        4. Step-by-step implementation plan (5-7 steps)""",
        expected_output="A structured project plan with features and tasks.",
        agent=project_manager
    )

    research_task = Task(
        description="""Based on the project plan, research and recommend the best tools and patterns.

        1. Best Python libraries to use (if any)
        2. Most suitable data structures
        3. Design patterns to apply
        4. Potential edge cases to handle
        5. Common pitfalls to avoid""",
        expected_output="Technical recommendations including library suggestions, data structures, and design patterns.",
        agent=researcher,
        context=[plan_task]
    )

```

```

)

code_task = Task(
    description="""Write complete, working Python code based on the following Requirements:
    Requirements:
    - Full implementation with all features
    - Proper error handling and input validation
    - Clear variable names and comments
    - Example usage at the bottom
    - PEP 8 compliant""",
    expected_output="Complete, runnable Python code with comments",
    agent=developer,
    context=[plan_task, research_task],
    output_file="solution.py"
)

test_task = Task(
    description="""Review the Python code and:
    1. Identify any bugs or logical errors
    2. Check edge case handling
    3. Write 5 pytest test cases covering:
        - Normal usage (happy path)
        - Edge cases (empty input, large input)
        - Error cases (invalid input)
    4. Suggest any improvements""",
    expected_output="Code review feedback + 5 pytest test cases",
    agent=tester,
    context=[code_task],
    output_file="test_solution.py"
)

doc_task = Task(
    description="""Create a professional README.md for this project. It should include:
    1. Project title and description
    2. Features list
    3. Installation instructions
    4. Usage examples with code snippets
    5. Function/class documentation
    6. Example output
    7. Contributing guidelines""",
    expected_output="Complete README.md in markdown format",
    agent=doc_writer,
    context=[code_task, test_task],
    output_file="README.md"
)

return [plan_task, research_task, code_task, test_task, doc_task]

```

— 3. Create and Run the Crew —

```
def main():
```

```

print("🏢 AI Software Company – CrewAI")
print("="*50)
print("Agents: Project Manager | Researcher | Developer | Tester")
print("="*50)

requirement = input("\n📝 Enter your software requirement: ")

if not requirement:
    requirement = "Build a Python library for managing a software project"
    print(f"\nUsing default: {requirement}")

tasks = create_tasks(requirement)

crew = Crew(
    agents=[project_manager, researcher, developer, tester],
    tasks=tasks,
    process=Process.sequential,
    verbose=True
)

print("\n🚀 Crew is working... (this may take 2-4 minutes)")
result = crew.kickoff()

print("\n" + "="*50)
print("✅ CREW COMPLETED!")
print("="*50)
print("\n📁 Generated Deliverables:")
print("  1. Code:      solution.py")
print("  2. Test Suite: test_solution.py")
print("  3. Docs:      README.md")
print("\n--- Final Documentation Preview ---")
print(result)

if __name__ == "__main__":
    main()

```

BASH

```
python ai_software_company.py
```

```
# Try these requirements:
```

```
# "Build a Python-based task manager with priority levels and a
```

```
# "Create a simple inventory management system for a small shop
```

```
# "Build a student attendance tracking system"
```



Watch agents think!

With verbose=True, see each agent's Thought process. The README.md is auto-saved and ready to push to GitHub.



Download Session 1 Python Code:



Download day3_01_ai_software_company.py



COMMON ISSUES & TROUBLESHOOTING

Error: CrewAI process task output missing

Fix: Specify output_file="output.md" inside your Task definition to ensure output is persisted to disk.



SESSION 1 CHECKPOINT & EXPECTED DELIVERABLES



Created 3 specialized CrewAI agents (PM, Developer, QA).



Sequential task execution generated a complete product spec and README.md.

SESSION 2 OF 3

🕒 11:15 AM – 1:00 PM

Build an Industry-Level AI Application

🏗️ CAPSTONE PROJECT OPTIONS



Industry Focus — Capstone Product Build

The Capstone Project simulates an

industry-level product sprint

🏗️. Student teams design, develop, and present full-stack AI solutions combining Streamlit UIs, LangChain/CrewAI logic, and vector stores for GitHub portfolio showcase.



Team Setup

Form teams of 3–4 students. Choose one project from below. Each team member owns one module. Use the standard architecture: Streamlit UI + LangChain + Ollama + ChromaDB.



PROGRAM ARCHITECTURE — FULL-STACK CAPSTONE STACK

**Streamlit Web UI****LangChain / CrewAI Engine****Ollama Local LLM****ChromaDB Vector Store****Real-Time Interactive Dashboard**



Option 1: AI Resume Analyzer

Stack: Streamlit + LangChain + RAG

- › Upload resume PDF
- › ATS compatibility score (out of 100)
- › Missing keywords for target role
- › Rewrite suggestions section-by-section
- › Compare against job description



Option 2: AI Interview Prep Assistant

Stack: Streamlit + LangChain + Agents

- › Input: role + company + resume
- › Generate role-specific questions
- › Evaluate student's answers
- › Provide model answers
- › Track weak topics



Option 3: College Admission Chatbot

Stack: Streamlit + RAG + LangChain

- › Load college brochure/handbook
- › Answer admission queries
- › Fee structure lookups
- › Course information
- › Chat with source citations



Option 4: Research Paper Assistant

Stack: Streamlit + RAG + LangGraph

- › Upload research papers (PDF)
- › Auto-summarize in plain English
- › Extract methodology & findings
- › Compare multiple papers
- › Generate literature review



Option 5: AI Code Reviewer

Stack: Streamlit + LangChain + CrewAI

- › Paste Python code
- › Bug detection and fixes
- › Code quality scoring
- › Performance suggestions
- › Auto-generate tests



Option 6: AI Study Planner

Stack: Streamlit + LangChain + Agents

- › Input: exam date + subjects + hours/day
- › Generate personalized study schedule
- › Daily topic recommendations
- › Progress tracking
- › Resource suggestions per topic



STANDARD PROJECT ARCHITECTURE (STREAMLIT UI)



Streamlit App Template

Use this as your starting point for any of the 6 projects above

1

Install Streamlit

BASH

```
pip install streamlit
```

PYTHON app.py

```

# Day 3 - Session 2: Capstone Project Template
# Replace [YOUR PROJECT] with your actual project logic
import streamlit as st
from langchain_ollama import ChatOllama
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

# — Page Configuration —
st.set_page_config(
    page_title="[Your Project Name]",
    page_icon="🤖",
    layout="wide",
    initial_sidebar_state="expanded"
)

# — Custom CSS —
st.markdown("""

""", unsafe_allow_html=True)

# — Sidebar —
with st.sidebar:
    st.title("⚙️ Settings")
    model_choice = st.selectbox(
        "LLM Model",
        ["llama3.2", "llama3.2:1b", "mistral"]
    )
    temperature = st.slider("Temperature", 0.0, 1.0, 0.3, 0.1)
    st.divider()
    st.markdown("**About**")
    st.markdown("Built with 🐙 Ollama + 🔗 LangChain")
    st.markdown("Agentic AI Workshop · Day 3")

# — Main UI —
st.title("🤖 [Your Project Name]")
st.markdown("*Brief description of what this app does*")
st.divider()

# — Initialize LLM —
@st.cache_resource
def get_llm(model, temp):
    return ChatOllama(model=model, temperature=temp)

llm = get_llm(model_choice, temperature)

```

```

# — Input Area —
col1, col2 = st.columns([2, 1])

with col1:
    # === CUSTOMIZE: Change to your input type ===
    user_input = st.text_area(
        "📝 Enter your input",
        placeholder="Type here...",
        height=200
    )
    # OR: uploaded_file = st.file_uploader("Upload PDF", type=

with col2:
    st.markdown("### 📖 How to use")
    st.markdown("""
1. Enter your input on the left
2. Click the button below
3. View AI analysis
""")

# — Process Button —
if st.button("🚀 Analyze", use_container_width=True):
    if not user_input.strip():
        st.warning("⚠️ Please enter some input first!")
    else:
        with st.spinner("🤖 AI is analyzing..."):
            # === CUSTOMIZE: Your LangChain logic here ===
            prompt = ChatPromptTemplate.from_messages([
                ("system", "You are a helpful AI assistant."),
                ("human", "Analyze this: {input}")
            ])
            chain = prompt | llm | StrOutputParser()
            result = chain.invoke({"input": user_input})

            # Display result
            st.success("✅ Analysis complete!")
            st.markdown("### 📄 Results")
            st.markdown(result)

            # Download button
            st.download_button(
                "📎 Download Report",
                data=result,
                file_name="ai_analysis.txt",
                mime="text/plain"
            )

# — Session History —
if "history" not in st.session_state:
    st.session_state.history = []

```

```
if st.session_state.history:
    st.divider()
    st.markdown("### 📄 Previous Analyses")
    for i, item in enumerate(reversed(st.session_state.history)):
        with st.expander(f"Analysis {i}"):
            st.write(item)
```

3

Run the Streamlit app

BASH

```
streamlit run app.py
# Opens browser at http://localhost:8501
```

4

GitHub Repository Setup

BASH

Terminal

```
# Initialize git and push to GitHub
git init
git add .
git commit -m "Initial commit: AI application capstone"
git branch -M main
git remote add origin https://github.com/yourusername/your-project
git push -u origin main
```

← Session 1

Session 3: Career Guidance →

Capstone Demo & Career Readiness

CAPSTONE PRESENTATION STRUCTURE



Production Focus — Cloud Deployment & Portfolios

Cloud Deployment

moves student applications from `localhost` to production platforms (Hugging Face / Render) , providing recruiters with live working URLs to verify technical competency.



Each team gets 10 minutes

: 7 min demo + 3 min Q&A. Follow this structure to make a strong impression.

1

Problem Statement (1 min)

- › What problem does your app solve?
- › Who is the target user?
- › Why does this problem matter?
- › One-line value proposition

2

Architecture (2 min)

- › Draw the system diagram on whiteboard
- › Which LangChain components used?
- › How does data flow through the system?
- › Why did you choose this architecture?

3

Live Demo (3 min)

- › Show the Streamlit UI
- › Use a real, interesting example
- › Highlight the AI reasoning
- › Show at least 2 different scenarios

4

GitHub Walk (1 min)

- › Show the repository
- › Point to the README
- › Briefly show code structure
- › Mention your contribution



Your Competitive Advantage

Most CSE graduates can write basic Python. You can now build real AI applications with RAG, Agents, and Multi-Agent Systems. This is what separates you from 90% of other candidates.



Top 20 AI Interview Q&A

Fundamentals

- › What is RAG and why is it needed?
- › Explain the difference between an LLM and an AI Agent
- › What is prompt engineering? Give 3 techniques
- › What is an embedding? How is it used in vector search?
- › Explain the ReAct pattern in AI agents

LangChain / Technical

- › What is LCEL and what's the pipe operator?
- › How does LangGraph differ from a regular LangChain agent?
- › What is ChromaDB and how do you query it?
- › How do you add memory to a LangChain chatbot?
- › Explain hallucination in LLMs and how to reduce it

Project-Based Questions

- › Walk me through your RAG pipeline
- › What challenges did you face with your AI project?
- › How would you scale this to 1000 users?
- › What would you add next to improve it?
- › How do you evaluate the quality of your RAG system?



Resume Bullet Points for AI Roles



Strong Resume Bullets

- › Built a RAG-based document Q&A system using LangChain + ChromaDB, reducing hallucinations by implementing hallucination grading
- › Developed a multi-agent AI system using CrewAI with 5 specialized agents (Researcher, Developer, Tester) collaborating sequentially
- › Created stateful AI workflows using LangGraph with conditional routing, memory persistence, and human-in-the-loop checkpoints
- › Deployed AI applications using Streamlit with LLM backend (Llama 3.2 via Ollama) — zero cloud cost



GitHub Portfolio Tips

- › Pin your 3 best AI repos on your profile
- › Every repo needs a detailed README with screenshots
- › Include a demo GIF or video in the README
- › Add requirements.txt for easy setup
- › Write clean commit messages (not "fix stuff")
- › Add topics/tags: #langchain #ai #python #rag

AI Certifications to Target

- › **DeepLearning.AI** — LangChain for LLM Application Development
- › **Coursera** — Generative AI with LLMs (AWS + DeepLearning.AI)
- › **Google** — Introduction to Generative AI (free)
- › **Hugging Face** — NLP Course (free)
- › **LangChain Academy** — Introduction to LangGraph

6-MONTH LEARNING ROADMAP

 17

Month 1–2: Consolidate

- › Complete all workshop projects from scratch
- › Push everything to GitHub with READMEs
- › Build one more RAG project (your topic)
- › Learn Streamlit in depth (multi-page apps)
- › Get Google Gen AI cert (free, 1 day)

 17

Month 3–4: Expand

- › Learn vector DB alternatives (Pinecone/FAISS)
- › Explore OpenAI API + cost management
- › Build one complete end-to-end project
- › Contribute to LangChain open source
- › Start applying for AI internships

 17

Month 5–6: Specialize

- › Pick a domain: healthcare / finance / education
- › Learn Docker basics (containerize your app)
- › Build a SaaS-style AI tool
- › Participate in AI hackathons
- › Apply for AI internship at product companies



Internship Hunt Strategy

Apply to startups building AI products (not just services). Look on LinkedIn for "AI Engineer Intern" with tags like LangChain, LLM, RAG. Your GitHub portfolio + these skills will get you shortlisted over candidates with no hands-on AI experience.



DAY 3 FINAL CHECKPOINT & PORTFOLIO DELIVERABLES



Full-stack AI application running locally on Streamlit.



Repository pushed to GitHub with clean README.md documentation.



Application deployed to cloud (Hugging Face Spaces / Render) with live public URL.

← Session 2



Back to Workshop Home



Congratulations on completing the 3-Day Agentic AI Workshop!

You are now equipped to build real AI applications. Go build something amazing! 🚀

← Day 1 · Day 2 · Workshop Home



Conducted by **Basavesh D.** · Dept. of CSE · Jyothy Institute of Technology, Bengaluru